



서울시립대학교

Lec 6 - UART (SCI)

전력전자연구실 교수 이 승 환



✓ UART 통신 - Universal Asynchronous Receiver / Transmitter)

- Serial 통신 방식의 가장 기본적인 형태
- MCU가 Rx 와 Tx 각 1개의 데이터 라인을 통해 한 비트씩 순서대로 데이터 주고 받는 방식
- Asynchronous 는 클럭에 동기화되어 있지 않고 아무때나 통신 하는 것, 대신 송수신자가 동일한 속도(baud rate)로 동작해야 함.
- Polling 또는 interrupt mode로 동작

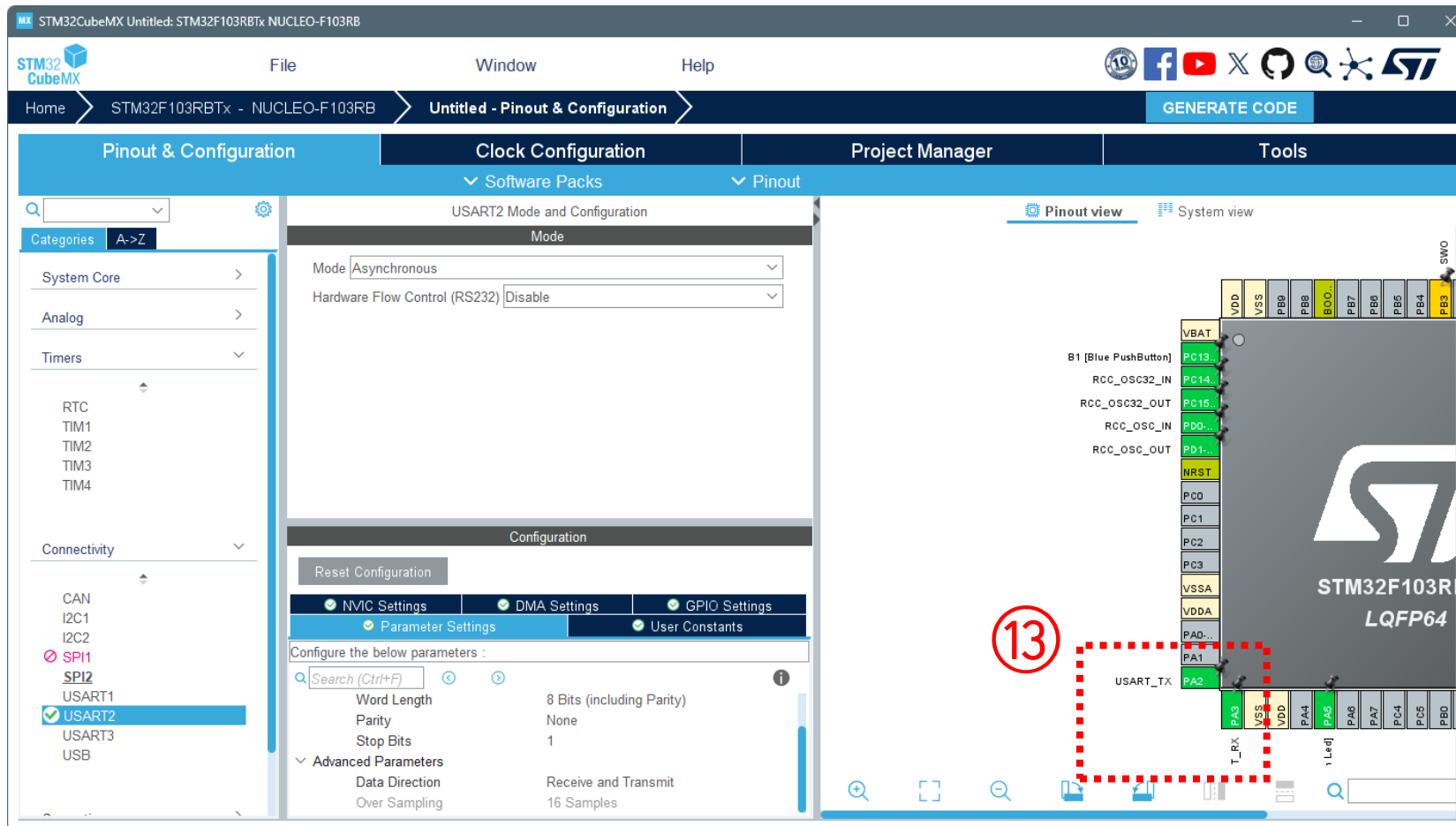
✓ Data frame 구조

- Start bit → 8 data bits → Parity bit (optional) → Stop bit

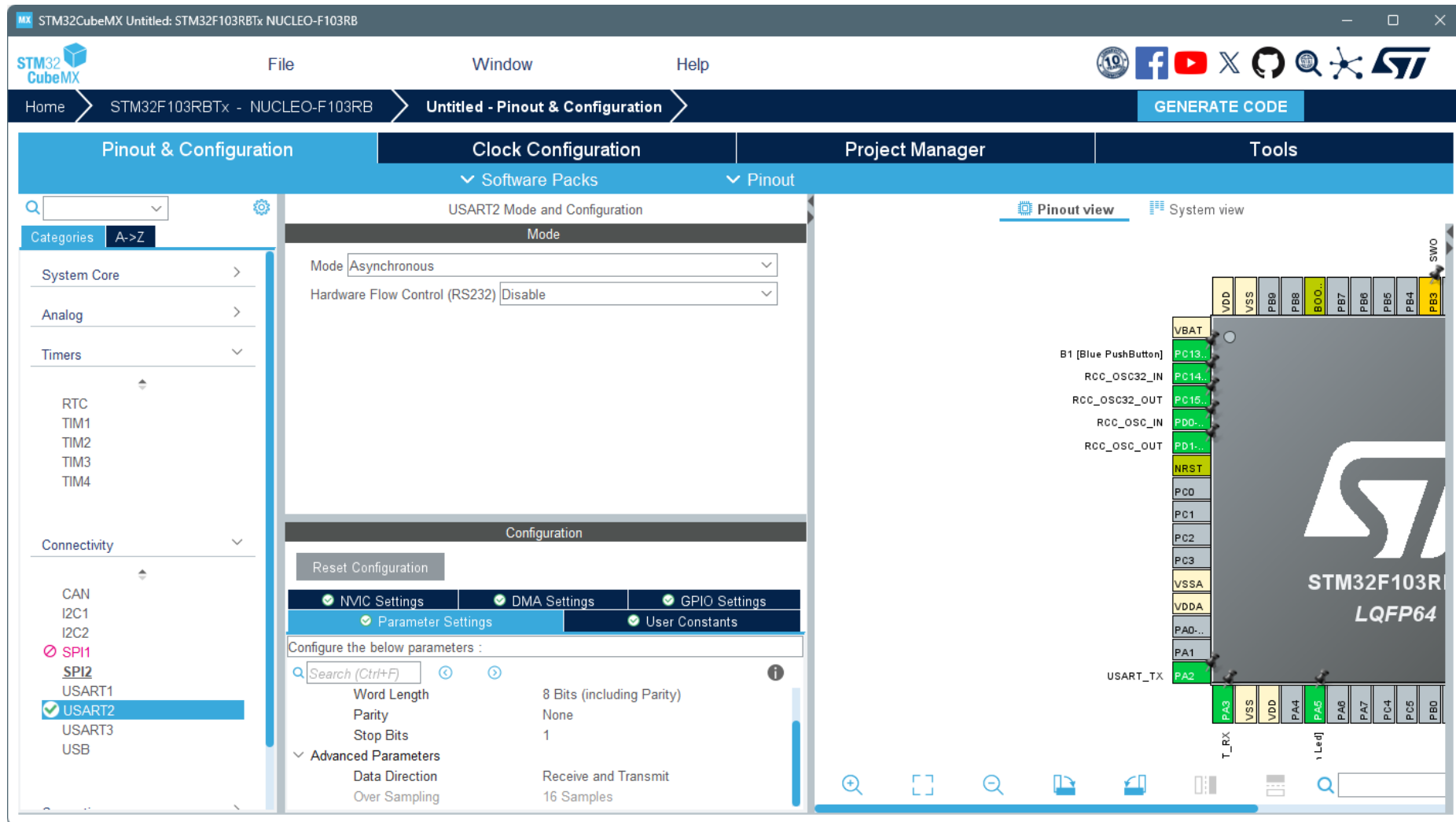
✓ 통신 설정 사항

- Baud rate : 초당 전송 비트 수 9600bps, 115200bps etc.
- Data bits: 데이터 비트 수, 7bit 또는 8bit
- Parity bit: None, Even, Odd
- Stop bits: 프레임 종료 신호, 1bit 또는 2bit

- 우리가 사용하는 MB1136보드는 ⑬번과 같이 PA2번 pin은 USART2 Tx pin으로 사용하고, PA3번 pin은 USART2 Rx pin으로 사용하도록 구성되어 있음



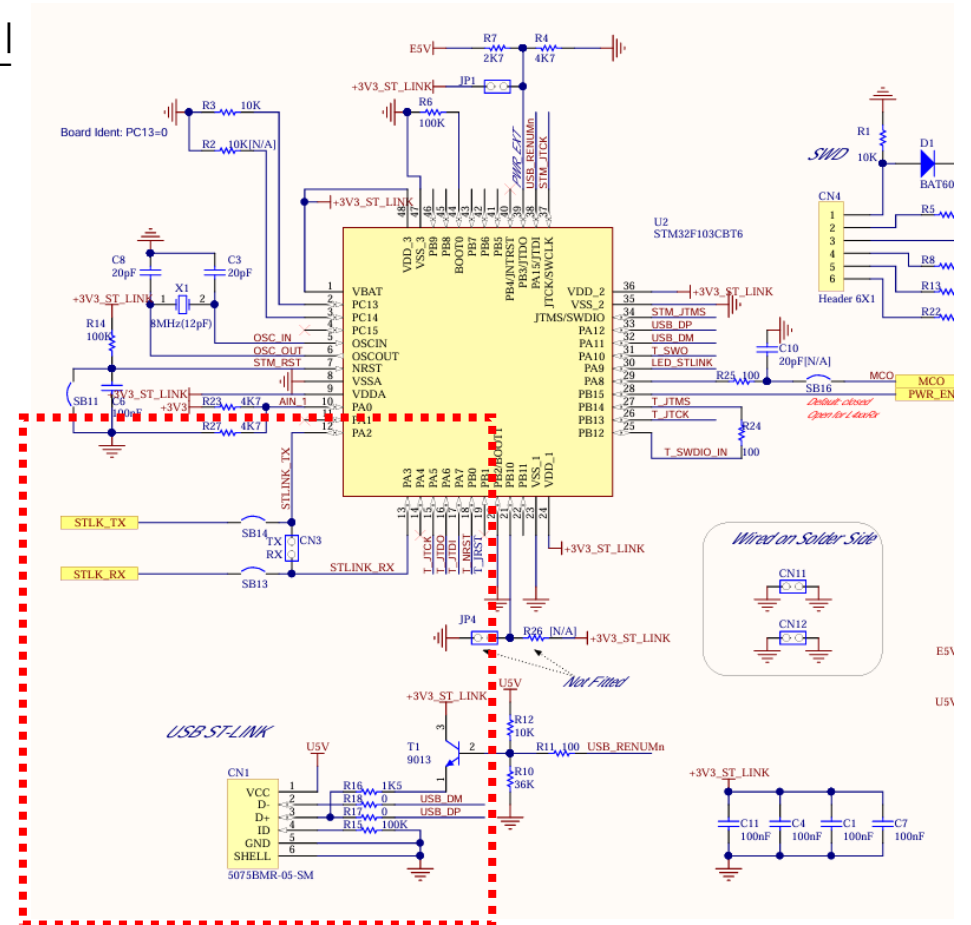
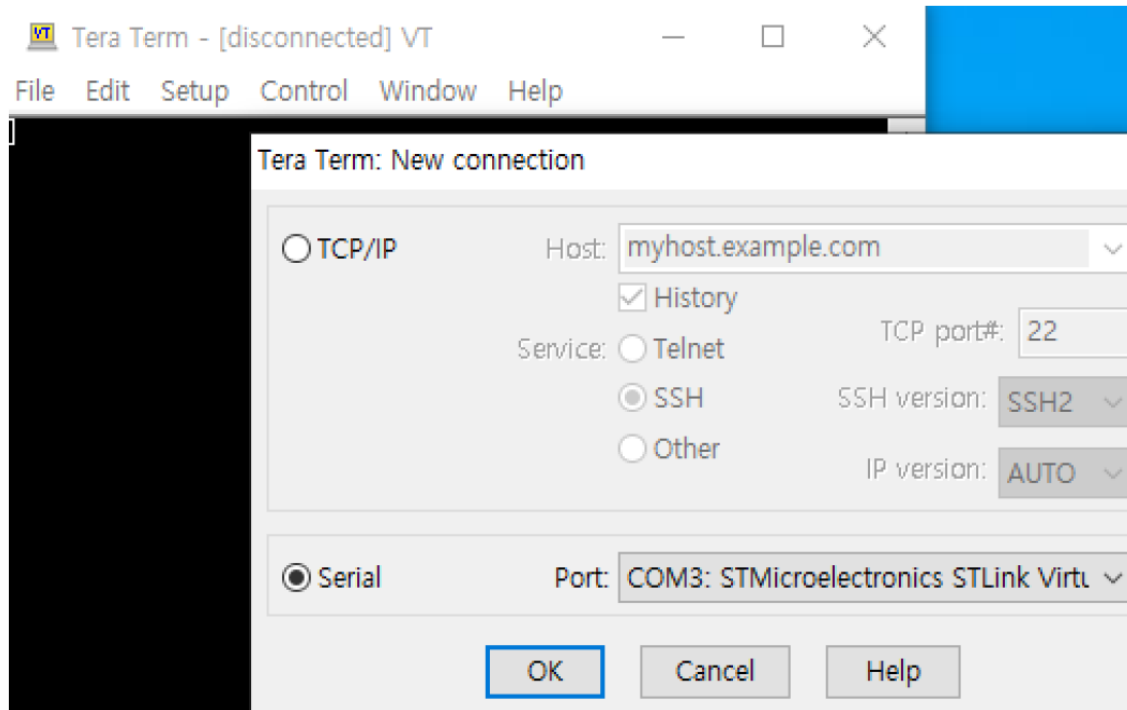
UART 설정 - 2



UART 문자 수신 실습 - 1.

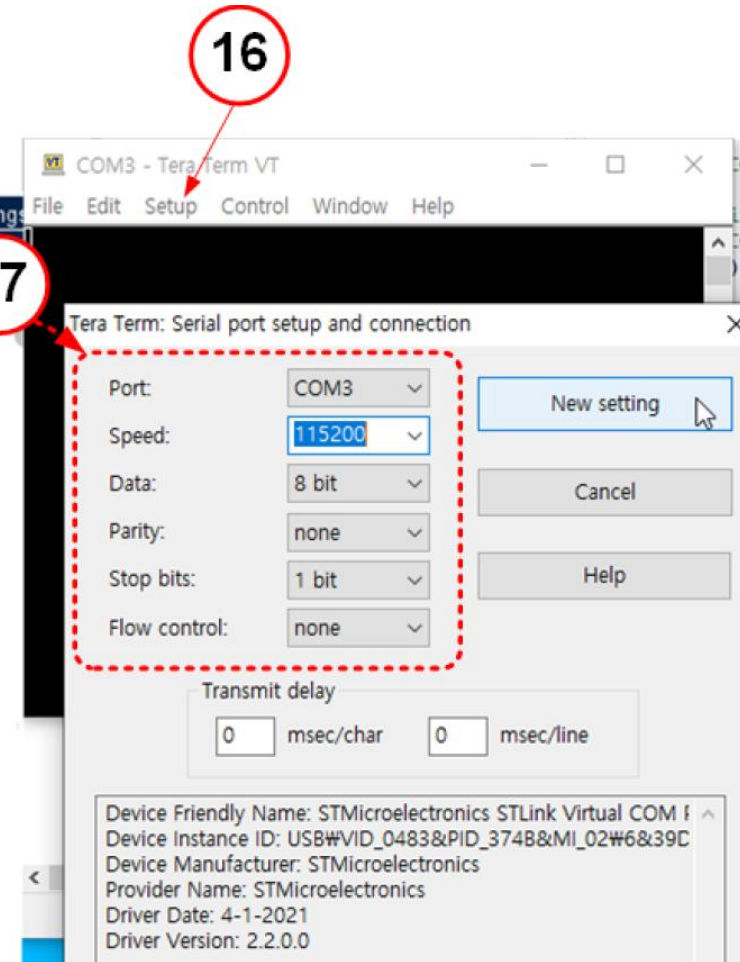
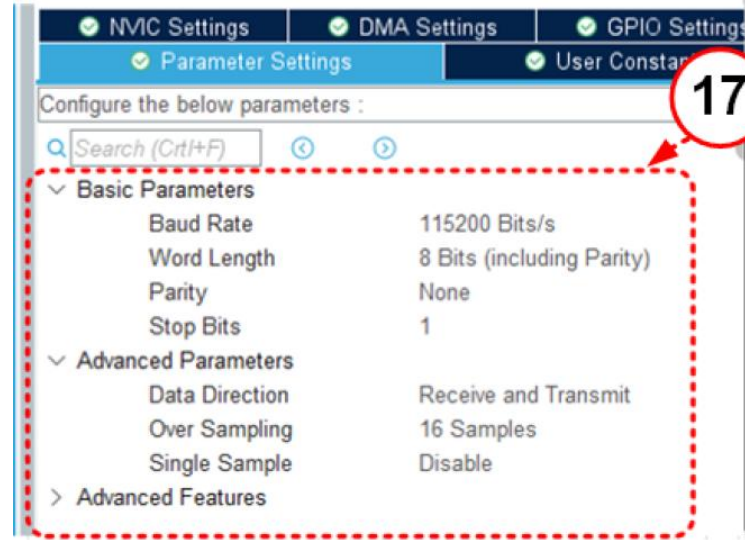
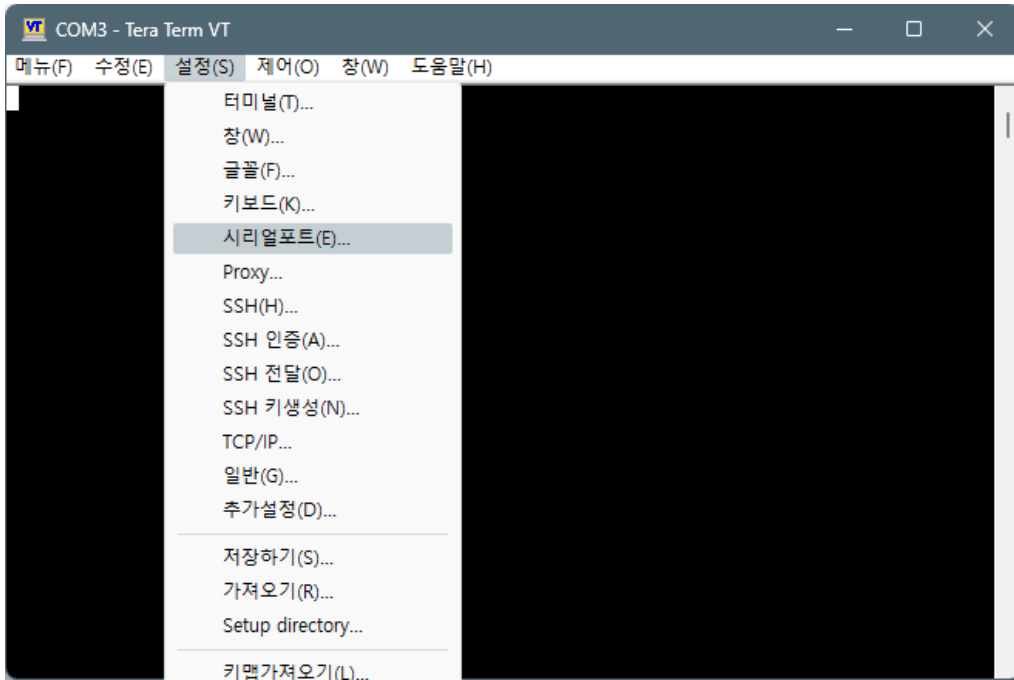
✓ TeraTerm 소프트웨어 설치 후 다음과 같이 COM Port 선택

- ST-Link는 USB 커넥터 통해서 IDE (IAR, CubeIDE 등)와 통신 + Virtual COM port 로 동작 가능 (컴퓨터 serial 통신 port)



UART 문자 수신 실습 - 2.

✓ 이후 TeraTerm → 설정 → 시리얼포트 설정 → 아래 그림과 같이 설정



UART 문자 수신 실습 - 3.

Tera Term: Terminal setup

Terminal size
80 X 24
☒ Term size = win size
☐ Auto window resize

Terminal ID: VT100

Answerback:

Coding (receive): UTF-8

Coding (transmit): UTF-8

locale: korean

New-line
Receive: LF
Transmit: CR
18
☒ Local echo
☐ Auto switch (VT<->TEK)

OK
Cancel
Help

UART 문자 수신 실습 - 4.

- Keyboard에서 “1” typing하여 주면, 문자 “1” 이 COM port를 통하여 연결되어 있는 Nucleo 보드의 USART2로 전송됨.
- 내부 UARTRxBuf 전역 변수에 ⑩번과 같이 문자 “1” 즉, ASCII code 값으로 49가 저장되는 것을 볼 수 있을 것.

```
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_USART2_UART_Init();
/* USER CODE BEGIN 2 */
HAL_UART_Receive_IT(&huart2, &UARTRxBuf, 1);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}
```

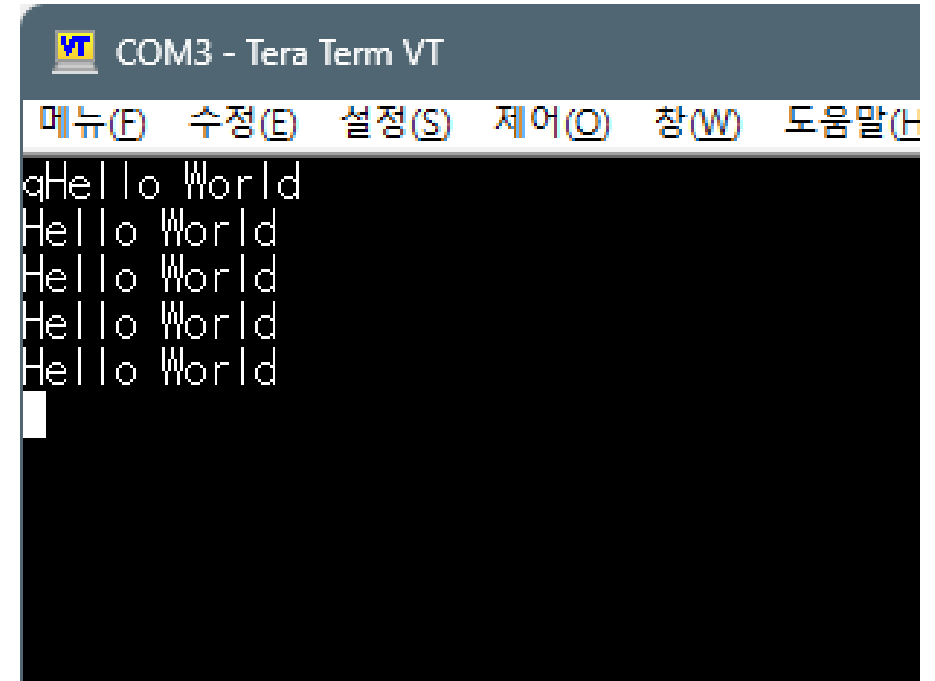
```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    int i;
    if(huart->Instance == USART2)
    {
        HAL_UART_Receive_IT(&huart2, &UARTRxBuf, 1);
    }
}
```

UART 문자 송신 실습 - 1.

- PC로 문자를 전송하기 위해서는 HAL_UART_Transmit() 함수를 사용하면 됨.
- Tera Term에 “q”를 typing하여 주면, 추가된 code 상에서 지정한 문자열이 gCnt 횟수만큼 출력되는 것을 확인할 수 있음.

```
/* USER CODE BEGIN PV */
uint8_t UARTRxBuf = 0;
uint8_t UARTTxBuf[] = "Hello World\n";
uint16_t gCnt = 0;
float fNum=0.1;
/* USER CODE END PV */

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    int i;
    if(huart->Instance == USART2)
    {
        if(UARTRxBuf == 'q'){
            while(gCnt<5){
                gCnt++;
                HAL_UART_Transmit(&huart2,UARTTxBuf,sizeof(UARTTxBuf),0xFFFF);
                for(i=1;i<100;i++){;}
            }
            gCnt = 0;
        }
        HAL_UART_Receive_IT(&huart2, &UARTRxBuf, 1);
    }
}
```



- 문자열을 전송 할 때에 형식화된 문자열 즉, stdio.h에 포함되어 있는 printf() 함수를 사용하면 편리.
- C언어에서 printf()는 “표준 출력(stdout)” 으로 데이터를 보냄.
- PC에서는 모니터 콘솔이 표준출력장치이나 MCU에서는 기본적으로 “표준 출력 장치”가 없음.
- 따라서 printf() 함수의 출력을 지정해줘야 함.
- IAR 에서는 printf() 함수를 call 하면 printf() → fputc() → __write() 함수 순으로 call해서 실행
- 따라서 fputc() 와 __write() 함수를 아래와 같이 정의해 줘야함.

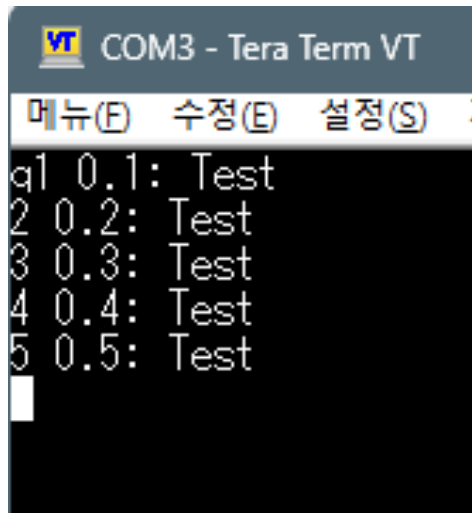
```
int fputc(int ch, FILE *f){
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, 0xFFFF);
    return ch;
}

size_t __write(int handle, const unsigned char * buffer, size_t size)
{
    if (handle != 1 && handle != 2) return -1; // stdout=1, stderr=2
    HAL_UART_Transmit(&huart2, (uint8_t*)buffer, size, HAL_MAX_DELAY);
    return size;
}
```

- printf() 함수 이용해서 출력하면 다음과 같이 나오게 할 수 있음.

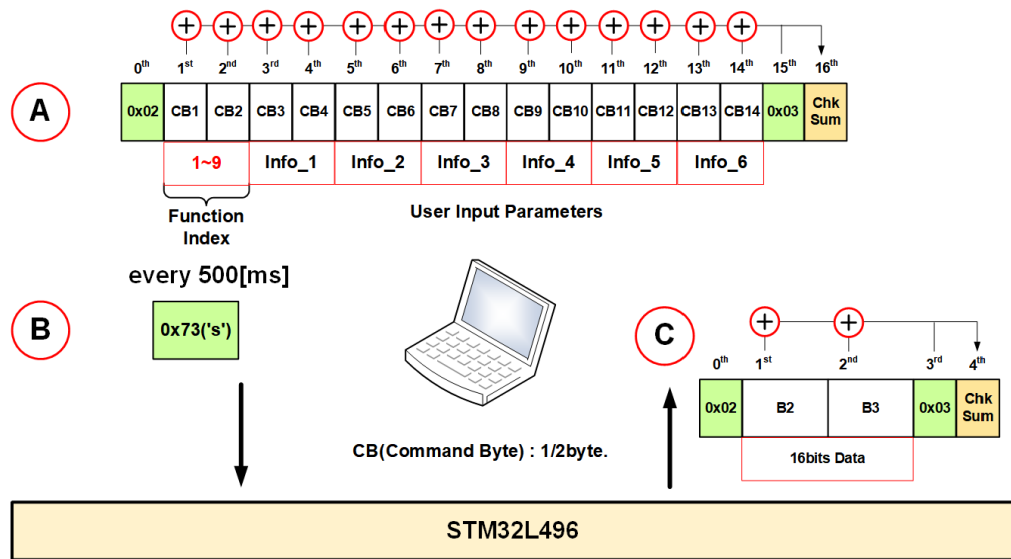
```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    int i;
    uint8_t gCnt = 0;
    float fNum = 0.0;

    if(huart->Instance == USART2)
    {
        if(UARTRxBuf == 'q'){
            while(gCnt<5){
                gCnt++;
                fNum = gCnt/10.0;
                printf("%d %.1f: Test\n",gCnt,fNum);
                for(i=1;i<100;i++){};
            }
        }
        HAL_UART_Receive_IT(&huart2, &UARTRxBuf, 1);
    }
}
```



✓ 통신 규약(protocol)은 상호 통신을 수행하기 위한 주체들 사이에 데이터를 주고 받기 위한 형식화된 약속을 의미

- 아래는 UART 를 이용한 일반 산업 현장에서 가장 많이 사용되는 간단한 통신 규약
- A는 Command Bytes (컴퓨터→MCU, 총 16 bytes)
- B는 매 500ms마다 PC가 단일 바이트 보내 Heartbeat나 Status 요청 신호 (컴퓨터→MCU)
- C는 MCU→컴퓨터로 보내는 data bytes (총 7 bytes)
- 대개는 송수신 하는 데이터는 Start of Text (STX, 0x02), Data bytes, End of Text(ETX, 0x03), ChkSum 의 형태로 구성



✓ UART 통신 기반 위에 프레임 구조 적용

- [STX][DATA_H][DATA_L][ETX][CHK]
- 예: 02 33 41 03 73 → 0x02 = STX, 0x33 0x41 = Data, 0x03 = ETX, 0x73 = ChkSum
- STX, ETX: 통신의 시작 및 끝 알려줘 프레임 단위 통신 진행할 수 있도록 함.
- 프레임 단위로 끊어져 있지 않으면 어디서 부터 어디까지가 데이터 인지 알 수 없음
- ChkSum: 데이터 검증용 코드(checksum)를 함께 보내 노이즈에 의한 데이터 오류 검출하고자 함.
- 다양한 ChkSum 생성 방법이 있지만, XOR 사용하는 것이 일반적 (구현 단순하고 연산속도 빠름)
- $02 \oplus 33 \oplus 41 \oplus 03 = 73$

✓ UART 활용 통신 프로토콜 수신 연습

- UART Receive IT complete callback 함수를 다음과 같이 수정
- 첫 byte가 0x02 (STX) 인지 체크하고 STX 이면 rxBuffer 에 저장하면서 rxIndex 하나 증가
- rxIndex > 0 이면 rxBuffer에 수신 바이트 저장
- 우리가 정의한 프로토콜은
[STX (1byte)] [Data_H(1byte)] [Data_L(1byte)] [ETX (1byte)] [Chksum (1byte)] 의 총 5 개의 데이터 이므로 rxIndex >= 4 되면 수신이 끝난 것.
- 따라서 rxFrameComplete 이라는 플래그를 1로 설정

```
/* USER CODE BEGIN PV */
uint8_t UARTRxBuf = 0;
uint8_t UARTRxBuf[] = "Hello World\n";
uint8_t txData[2];
uint8_t rxByte;
uint8_t rxBuffer[5];
uint8_t rxIndex = 0;
uint8_t rxFrameComplete = 0;
/* USER CODE END PV */
```

```
/* Private function prototypes -----
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
/* USER CODE BEGIN PFP */
void sendDataFrame(uint8_t *data);
uint8_t calcChecksum(uint8_t *buf, uint8_t len);
void processReceivedFrame(uint8_t *frame, uint8_t len);
/* USER CODE END PFP */
```

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    int i;
    uint8_t gCnt = 0;
    float fNum = 0.0;

    if(huart->Instance == USART2)
    {
        uint8_t b = rxByte;
        // printf("RX: 0x%02X\r\n", rxByte);

        //received data fram parsing
        if (b == 0x02 && rxIndex == 0) // STX
        {
            rxBuffer[rxIndex++] = b;
        }
        else if (rxIndex > 0)
        {
            rxBuffer[rxIndex++] = b;
            if (rxIndex >= sizeof(rxBuffer)) rxIndex = 0; // overflow 방지

            if (rxIndex >= 4) {
                rxFrameComplete = 1;
            }
        }
    }

    HAL_UART_Receive_IT(&huart2, &rxByte, 1);
    /*
    if(UARTRxBuf == 'q'){
        while(gCnt<5){
            gCnt++;
            fNum = gCnt/10.0;
            printf("%d %.1f: Test\n",gCnt,fNum);
            for(i=1;i<100;i++){
            }
        }
    }
    HAL_UART_Receive_IT(&huart2, &UARTRxBuf, 1);
    */
}
```

✓ main() 의 while() 함수는 다음과 같이 수정하고

```

/* USER CODE BEGIN 2 */
// HAL_UART_Receive_IT(&huart2, &UARTRxBuf, 1);
HAL_UART_Receive_IT(&huart2, &rxByte, 1);

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    if (rxFrameComplete){
        processReceivedFrame(rxBuffer, rxIndex);
        rxFrameComplete = 0;
        rxIndex = 0;
    }

    HAL_Delay(100);
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

```

✓ 다음과 같은 수신 데이터
postprocessing 함수인
processReceivedFrame() 작성

- Length (len) 이 5보다 작으면
wrong frame warning 띄움
- STX 와 ETX 오류 있으면
wrong frame structure warning
- calcChecksum 함수 이용해 수신한
ChkSum과 연산한 ChkSum 동일한지 확인
- 수신데이터가 프로토콜과 맞으면 마지막에
printf 로 Teraterm에 received data 를 다시
송신해서 보여줌

```
uint8_t calcChecksum(uint8_t *buf, uint8_t len)
{
    uint8_t sum = 0;
    for (uint8_t i = 0; i < len; i++)
        sum ^= buf[i];
    return sum;
}
```

```
void processReceivedFrame(uint8_t *frame, uint8_t len)
{
    // 구조: [0]=STX [1]=ASCII1 [2]=ASCII2 [3]=ETX [4]=CHK
    if (len < 5){
        printf("Wrong frame (too short)\r\n");
        return;
    }

    if (frame[0] != 0x02 || frame[3] != 0x03) {
        printf("Wrong frame structure\r\n");
        return;
    }

    uint8_t checksum = calcChecksum(frame, 4);
    if (checksum != frame[4])
    {
        printf("Checksum error! rx=0x%02X calc=0x%02X\r\n", frame[4], checksum);
        return;
    }

    printf("Received data = (%x%x)\r\n", frame[1], frame[2]);
}
```

- ✓ Teraterm 의 매크로 기능 활용해
검증하거나 오른쪽과 같은 파이썬 코드
이용해서 검증 가능
 - Teraterm 이용하는 경우 아래와 같은
auto_send_frame.ttl 파일을 메모장 등을
이용해 만들어서 Teraterm의 매크로 기능 활용
 - 파이썬 코드 이용하는 경우 오른쪽 코드 vscode
이용해 작성하고 실행해 02 33 41 03 등의
[STX][Data1][Data2][ETX] 포맷으로 데이터
입력해주면 ChkSum을 자동으로 계산해서
Nucleo 보드에 송신함
 - 올바른 포맷의 데이터 보내면
Received data = 형태의
답 받을 수 있음
 - 올바르지 않은 포맷의 데이터
도 시험해 볼것

```

; auto_send_frame.ttl
:loop
send $02
send $33
send $45
send $03
send $77
mpause 500
goto loop
    
```

```

import serial, threading

s = serial.Serial('COM3', 115200, timeout=0.1)

def read_from_board():
    while True:
        if s.in_waiting > 0:
            data = s.read(s.in_waiting)
            print(data.decode(errors='ignore'), end='')

threading.Thread(target=read_from_board, daemon=True).start()

def xor_checksum(data):
    chk = 0
    for b in data:
        chk ^= b
    return chk

print("NUCLEO 통신 시작됨.")
print("HEX 값 입력 하세요 (예: 02 33 41 03) → checksum 자동 추가. 빈 줄 입력 시 종료.\n")

while True:
    cmd = input("HEX> ")
    if not cmd:
        break
    try:
        data = bytes(int(x, 16) for x in cmd.split())
        chk = xor_checksum(data)
        frame = data + bytes([chk])
        print(f"전송 프레임: {[hex(x) for x in frame]}")
        s.write(frame)
    except Exception as e:
        print("오류:", e)
    
```

✓ UART 활용 통신 프로토콜에 맞춘 데이터 송신

- 다음과 같은 sendFrame 함수 추가하고
- main()의 while()문을 다음과 같이 수정
- txData로 인해 TeraTerm에 어떤 데이터가 나오는지 확인

```
void sendDataFrame(uint8_t *data)
{
    uint8_t frame[6];

    frame[0] = 0x02;
    frame[1] = data[0];
    frame[2] = data[1];
    frame[3] = 0x03;
    frame[4] = calcChecksum(frame, 4);

    HAL_UART_Transmit(&huart2, frame, 5, 100);
}
```

```
/* USER CODE BEGIN WHILE */
txData[0] = 0x01;
txData[1] = 0x02;
while (1)
{
    sendDataFrame(txData);
    txData[0]++;
    txData[1]++;

    /* if (rxFrameComplete){
        processReceivedFrame(rxBuffer, rxIndex);
        rxFrameComplete = 0;
        rxIndex = 0;
    } */

    HAL_Delay(100);
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}
```

- ✓ 과제1: LED 제어 프로토콜 구현 (LD2 ON/OFF 조교확인 필수)
 - UART 통신을 이용해 명령 기반 제어 프로토콜(command frame) 을 직접 설계
 - [STX][CMD][DATA][ETX][CHK]
 - CMD = 0x31 ('1') → LED ON
 - CMD = 0x30 ('0') → LED OFF
 - DATA = 0x00 (dummy, reserved)
 - 예:
 - LED On: 02 31 30 03 64 → GPIO control to LD2 ON
 - LED Off: 02 30 30 03 64 → GPIO control to LD2 ON
- ✓ 과제2: 과제 1의 CMD를 3개의 명령(ON/OFF/TOGGLE)으로 확장하고,
MCU가 수행 결과를 “ACK(0x06)” 또는 “NAK(0x15)” 프레임으로 응답하도록 수정 하고 LD2 ON/OFF결과와 TeraTerm에서 ACK또는 NAK 조교확인필수